

Göz bebeklerinin tespit edilmesi

```
try:

    isolate_left = isolate_eye(frame, landmarks, [36, 37, 38, 39, 40, 41])[0]
    isolate_right = isolate_eye(frame, landmarks, [42, 43, 44, 45, 46, 47])[0]

    isolate_left_gray = cv2.cvtColor(isolate_left, cv2.COLOR_BGR2GRAY)
    isolate_right_gray = cv2.cvtColor(isolate_right, cv2.COLOR_BGR2GRAY)

    isolate_left_blur = cv2.medianBlur(isolate_left_gray, 5)
    isolate_right_blur = cv2.medianBlur(isolate_right_gray, 5)

    leftEyeCenterCoordinate = eyeMidCenter([37,38,40,41],landmarks)

    leftEyeCenterCoordinate = (
    leftEyeCenterCoordinate[0] - isolate_eye(frame, landmarks, [36, 37, 38, 39, 40, 41])[1],
    leftEyeCenterCoordinate[1] - isolate_eye(frame, landmarks, [36, 37, 38, 39, 40, 41])[2])

    rightEyeCenterCoordinate = eyeMidCenter([43,44,46,47], landmarks)
    rightEyeCenterCoordinate = (
    rightEyeCenterCoordinate[0] - isolate_eye(frame, landmarks, [42, 43, 44, 45, 46, 47])[1],
    rightEyeCenterCoordinate[1] - isolate_eye(frame, landmarks, [42, 43, 44, 45, 46, 47])[2])

except:
    pass

try:

    param_1 = 200
    param_2 = 5
    maxradius = 8
    minradius = 1

    leftEyePupils = cv2.HoughCircles(isolate_left_blur, cv2.HOUGH_GRADIENT, 1, 999, param1=param_1,
                                     param2=param_2, maxRadius=maxradius, minRadius=minradius)

    leftEyePupils = np.uint16(np.around(leftEyePupils))

    for i in leftEyePupils[0, :]:
        # dış daire çizimi
        leftEyeCircle = cv2.circle(isolate_left, (i[0], i[1]), i[2], green, 1)
        # iç daire çizimi
        leftEyeCircleIn = cv2.circle(isolate_left, (i[0], i[1]), 2, red, 1)

        leftEyePupilX = (i[0])
        leftEyePupilY = (i[1])

        leftEyePupilCoordinate = (leftEyePupilX, leftEyePupilY)

        leftEyeFrameResize = cv2.resize(isolate_left, None, fx=10, fy=10)

        # cv2.imshow("leftEyePupil", leftEyeFrameResize)

        # leftEyeCoordinate = ("Left Eye Pupil Cor: " + str(leftEyePupilCoordinate))

except:
    pass

try:

    param_1 = 200
    param_2 = 5
    maxradius = 8
    minradius = 1

    rightEyePupils = cv2.HoughCircles(isolate_right_blur, cv2.HOUGH_GRADIENT, 1, 999,
                                     param1=param_1,
                                     param2=param_2, maxRadius=maxradius, minRadius=minradius)

    rightEyePupils = np.uint16(np.around(rightEyePupils))

    for a in rightEyePupils[0, :]:
        # dış daire çizimi
        cv2.circle(isolate_right, (a[0], a[1]), a[2], green, 1)
        # iç daire çizimi
        cv2.circle(isolate_right, (a[0], a[1]), 2, red, 1)

        rightEyePupilX = (a[0])
        rightEyePupilY = (a[1])

        rightEyePupilCoordinate = (rightEyePupilX, rightEyePupilY)

        rightEyeFrameResize = cv2.resize(isolate_right, None, fx=10, fy=10)

        # cv2.imshow("rightEyePupil", rightEyeFrameResize)

        # rightEyeCoordinate = ("Right Eye Pupil Cor: " + str(rightEyePupilCoordinate))

except:
    pass
```

Kısayolların Çalışmasını Sağlayan Kodlar

```
if np.all((leftEyePupils == None)) and np.all(
    (rightEyePupils == None)) and blinking_ratio > 5.7 and len(
    pause_set) == 0 and eye_brow == 0:
    pause_blinking += 1
    pause_now = datetime.datetime.now()
    pause_seconds = time.mktime(pause_now.timetuple())
    pause_seconds = int(pause_seconds)
    pause_time_list.append(pause_seconds)
    if pause_blinking >= 50 and (int(pause_time_list[-1]) - int(pause_time_list[0])) <= 5:
        pause_set.add(1)
        pause_blinking = 0
        pause_time_list.clear()
        playsound("./tools/stop.mp3")
    elif pause_blinking < 50 and (int(pause_time_list[-1]) - int(pause_time_list[0])) > 5:
        pause_blinking = 0
        pause_time_list.clear()

if np.all((leftEyePupils == None)) and np.all(
    (rightEyePupils == None)) and blinking_ratio > 5.7 and len(
    pause_set) == 1 and eye_brow == 0:
    continue_blinking += 1
    continue_now = datetime.datetime.now()
    continue_seconds = time.mktime(continue_now.timetuple())
    continue_seconds = int(continue_seconds)
    continue_time_list.append(continue_seconds)
    if continue_blinking >= 50 and (int(continue_time_list[-1]) - int(continue_time_list[0])) <= 5:
        pause_set.clear()
        continue_time_list.clear()
        continue_blinking = 0
        playsound("./tools/start.mp3")
    elif continue_blinking < 50 and (int(continue_time_list[-1]) - int(continue_time_list[0])) > 5:
        continue_blinking = 0
        continue_time_list.clear()

if len(pause_set) == 0:

    if (left_eye_ratio > blinking_ratio and left_eye_ratio > 5.7 and np.all(leftEyePupils == None)
        and np.all(rightEyePupils != None) and eye_brow == 0):
        desktop_blinking += 1
        desktop_now = datetime.datetime.now()
        desktop_seconds = time.mktime(desktop_now.timetuple())
        desktop_seconds = int(desktop_seconds)
        desktop_time_list.append(desktop_seconds)
        if desktop_blinking >= 50 and (int(desktop_time_list[-1]) - int(desktop_time_list[0])) <= 5:
            desktop_time_list.clear()
            desktop_blinking = 0
            pyautogui.hotkey("win", "d")
            playsound("./tools/desktop.mp3")
        elif desktop_blinking < 50 and (int(desktop_time_list[-1]) - int(desktop_time_list[0])) > 5:
            desktop_blinking = 0
            desktop_time_list.clear()

    if (right_eye_ratio > blinking_ratio and right_eye_ratio > 5.7 and np.all(rightEyePupils == None)
        and np.all(leftEyePupils != None) and eye_brow == 0):
        keyboard_blinking += 1
        keyboard_now = datetime.datetime.now()
        keyboard_seconds = time.mktime(keyboard_now.timetuple())
        keyboard_seconds = int(keyboard_seconds)
        keyboard_time_list.append(keyboard_seconds)
        if keyboard_blinking >= 25 and (int(keyboard_time_list[-1]) - int(keyboard_time_list[0])) <= 5:
            keyboard_time_list.clear()
            keyboard_blinking = 0
            pyautogui.hotkey("win", "ctrl", "o")
            playsound("./tools/keyboard.mp3")
        elif keyboard_blinking < 25 and (int(keyboard_time_list[-1]) - int(keyboard_time_list[0])) > 5:
            keyboard_blinking = 0
            keyboard_time_list.clear()

    if np.all((leftEyePupils == None)) and np.all(
        (rightEyePupils == None)) and blinking_ratio > 5.7 and eye_brow == 1:
        file_blinking += 1
        file_now = datetime.datetime.now()
        file_seconds = time.mktime(file_now.timetuple())
        file_seconds = int(file_seconds)
        file_time_list.append(file_seconds)
        if file_blinking >= 30 and (int(file_time_list[-1]) - int(file_time_list[0])) <= 5:
            file_time_list.clear()
            file_blinking = 0
            pyautogui.hotkey("win", "e")
            playsound("./tools/file.mp3")
        elif file_blinking < 30 and (int(file_time_list[-1]) - int(file_time_list[0])) > 5:
            file_blinking = 0
            file_time_list.clear()
```

Mod 1 ve Mod 2 seçeneklerinin çalışmasını sağlayan kod satırları - 1

```
left_eye_ratio = get_blinking_ratio([36, 37, 38, 39, 40, 41], landmarks)
right_eye_ratio = get_blinking_ratio([42, 43, 44, 45, 46, 47], landmarks)
blinking_ratio = round((left_eye_ratio + right_eye_ratio) / 2, 2)

eye_brow = eyebrow([21, 22, 27, 0, 1, 16, 15], landmarks)

while modChoiceEx == 0 and secondPass > 0:

    if (left_eye_ratio > blinking_ratio and left_eye_ratio > 5.7 and np.all(leftEyePupils == None)
        and np.all(rightEyePupils != None)):
        mod1Choice += 1
    elif (right_eye_ratio > blinking_ratio and right_eye_ratio > 5.7 and np.all(rightEyePupils == None)
        and np.all(leftEyePupils != None)):
        mod2Choice += 1
    break

if mod1Choice > 1 or mod2Choice > 1:
    modChoiceEx += 1

if mod1Choice >= 2 and (mod2Choice == 1 or mod2Choice == 0):

    noiseAreaX = int((landmarks.part(16).x - landmarks.part(0).x) // 10)
    noiseAreaY = int((landmarks.part(12).y - landmarks.part(0).y) // 2)

    noiseAreaStart = (landmarks.part(0).x - noiseAreaX, landmarks.part(0).y - noiseAreaY)
    noiseAreaEnd = (landmarks.part(16).x + noiseAreaX, landmarks.part(12).y + noiseAreaY)

    zeroPointX = ((noiseAreaEnd[0] + noiseAreaStart[0]) // 2)
    zeroPointY = ((noiseAreaStart[1] + noiseAreaEnd[1]) // 2)

    cv2.rectangle(frame, noiseAreaStart, noiseAreaEnd, red, 2)

    noise = (landmarks.part(30).x, landmarks.part(30).y)

    cv2.circle(frame, noise, 2, blue, -1)
    cv2.circle(frame, (zeroPointX, zeroPointY), 2, red, -1)

    safeAreaX = int((noiseAreaEnd[0] - noiseAreaStart[0]) // 8)
    safeAreStart = (zeroPointX - safeAreaX, zeroPointY - safeAreaX)
    safeAreEnd = (zeroPointX + safeAreaX, zeroPointY + safeAreaX)

    cv2.rectangle(frame, safeAreStart, safeAreEnd, blue, 2)

    if np.all((leftEyePupils == None)) and np.all(
        (rightEyePupils == None)) and blinking_ratio > 5.7 and len(
        pause_set) == 0 and eye_brow == 0:....

    if np.all((leftEyePupils == None)) and np.all(
        (rightEyePupils == None)) and blinking_ratio > 5.7 and len(
        pause_set) == 1 and eye_brow == 0:....

    if len(pause_set) == 0:

        if (left_eye_ratio > blinking_ratio and left_eye_ratio > 5.7 and np.all(leftEyePupils == None)
            and np.all(rightEyePupils != None) and eye_brow == 0):....

        if (right_eye_ratio > blinking_ratio and right_eye_ratio > 5.7 and np.all(rightEyePupils == None)
            and np.all(leftEyePupils != None) and eye_brow == 0):....

        if np.all((leftEyePupils == None)) and np.all(
            (rightEyePupils == None)) and blinking_ratio > 5.7 and eye_brow == 1:....

        if (zeroPointY - safeAreaX > noise[1]):
            cursorMod1 = noise[1] - (zeroPointY - safeAreaX)
            position = pygameui.position()
            pygameui.moveTo(position[0], position[1] + (2 * cursorMod1))
        elif (noise[1] > zeroPointY + safeAreaX):
            cursorMod1 = (zeroPointY + safeAreaX) - noise[1]
            position = pygameui.position()
            pygameui.moveTo(position[0], position[1] - (2 * cursorMod1))
        elif (noise[0] > zeroPointX + safeAreaX):
            cursorMod1 = noise[0] - (zeroPointX + safeAreaX)
            position = pygameui.position()
            pygameui.moveTo(position[0] + (2 * cursorMod1), position[1])
        elif (zeroPointX - safeAreaX > noise[0]):
            cursorMod1 = (zeroPointX - safeAreaX) - noise[0]
            position = pygameui.position()
            pygameui.moveTo(position[0] - (2 * cursorMod1), position[1])

    mod1a = (left_eye_ratio > blinking_ratio and left_eye_ratio > 5.7 and np.all(leftEyePupils == None)
        and np.all(rightEyePupils != None))
    mod1b = (right_eye_ratio > blinking_ratio and right_eye_ratio > 5.7 and np.all(rightEyePupils == None)
        and np.all(leftEyePupils != None))

    if mod1a and (noise[0] > safeAreStart[0] and noise[0] < safeAreEnd[0]) and (
        noise[1] > safeAreStart[1] and noise[1] < safeAreEnd[1]):
        pygameui.click(button="left")
    if mod1b and (noise[0] > safeAreStart[0] and noise[0] < safeAreEnd[0]) and (
        noise[1] > safeAreStart[1] and noise[1] < safeAreEnd[1]):
        pygameui.click(button="right")
```

Mod 1 ve Mod 2 seçeneklerinin çalışmasını sağlayan kod satırları - 2

```
try:

    rightlookX = int(((leftEyePupilCoordinate[0] - leftEyeCenterCoordinate[0]) + (
        rightEyePupilCoordinate[0] - rightEyeCenterCoordinate[0])) // 2)
    leftlookX = int(((leftEyeCenterCoordinate[0] - leftEyePupilCoordinate[0]) + (
        rightEyeCenterCoordinate[0] - rightEyePupilCoordinate[0])) // 2)
    uplookX = int(((leftEyeCenterCoordinate[1] - leftEyePupilCoordinate[1]) + (
        rightEyeCenterCoordinate[1] - rightEyePupilCoordinate[1])) // 2)

    eye_pupil_x = int((rightEyePupilCoordinate[0] + leftEyePupilCoordinate[0]) // 2)
    eye_center_x = int((rightEyeCenterCoordinate[0] + leftEyeCenterCoordinate[0]) // 2)
    eye_pupil_y = int((rightEyePupilCoordinate[1] + leftEyePupilCoordinate[1]) // 2)
    eye_center_y = int((rightEyeCenterCoordinate[1] + leftEyeCenterCoordinate[1]) // 2)

    hor_line_lenght = int(hor([36, 39, 42, 45], landmarks) // 8)

    safeEyeAreaStart = (eye_center_x - hor_line_lenght, eye_center_y - hor_line_lenght)
    safeEyeAreaEnd = (eye_center_x + hor_line_lenght, eye_center_y + hor_line_lenght)

    if mod2Choice >= 2 and (mod1Choice == 1 or mod1Choice == 0):

        noiseAreaX = int((landmarks.part(16).x - landmarks.part(0).x) // 16)
        noiseAreaY = int((landmarks.part(12).y - landmarks.part(0).y) // 2)

        noiseAreaStart = (landmarks.part(0).x - noiseAreaX, landmarks.part(0).y - noiseAreaY)
        noiseAreaEnd = (landmarks.part(16).x + noiseAreaX, landmarks.part(12).y + noiseAreaY)

        zeroPointX = ((noiseAreaEnd[0] + noiseAreaStart[0]) // 2)
        zeroPointY = ((noiseAreaStart[1] + noiseAreaEnd[1]) // 2)

        cv2.rectangle(frame, noiseAreaStart, noiseAreaEnd, red, 2)

        noise = (landmarks.part(30).x, landmarks.part(30).y)

        cv2.circle(frame, noise, 2, blue, -1)
        cv2.circle(frame, (zeroPointX, zeroPointY), 2, red, -1)

        if np.all((leftEyePupils == None)) and np.all(
            (rightEyePupils == None)) and blinking_ratio > 5.7 and len(
            pause_set) == 0 and eye_brow == 0:...

        if np.all((leftEyePupils == None)) and np.all(
            (rightEyePupils == None)) and blinking_ratio > 5.7 and len(
            pause_set) == 1 and eye_brow == 0:...

        if len(pause_set) == 0:

            if (left_eye_ratio > blinking_ratio and left_eye_ratio > 5.7 and np.all(leftEyePupils == None)
                and np.all(rightEyePupils != None) and eye_brow == 0):...

            if (right_eye_ratio > blinking_ratio and right_eye_ratio > 5.7 and np.all(rightEyePupils == None)
                and np.all(leftEyePupils != None) and eye_brow == 0):...

            if np.all((leftEyePupils == None)) and np.all(
                (rightEyePupils == None)) and blinking_ratio > 5.7 and eye_brow == 1:...

            if np.all((leftEyePupils != None)) and np.all((rightEyePupils != None)) and len(
                pause_set) == 0:
                if (eye_pupil_y < safeEyeAreaStart[1]):
                    cursorMod2 = 6 * uplookX
                    position = pyautogui.position()
                    pyautogui.moveTo(position[0], position[1] - cursorMod2)
                elif (eye_brow == 1):
                    cursorMod2 = 15
                    position = pyautogui.position()
                    pyautogui.moveTo(position[0], position[1] + cursorMod2)
                elif (eye_pupil_x < safeEyeAreaStart[0]):
                    cursorMod2 = 6 * leftlookX
                    position = pyautogui.position()
                    pyautogui.moveTo(position[0] - cursorMod2, position[1])
                elif (eye_pupil_x > safeEyeAreaEnd[0]):
                    cursorMod2 = 6 * rightlookX
                    position = pyautogui.position()
                    pyautogui.moveTo(position[0] + cursorMod2, position[1])

            if np.all((leftEyePupils == None)) or np.all((rightEyePupils == None)):
                if (left_eye_ratio > blinking_ratio and left_eye_ratio > 5.7 and np.all(leftEyePupils == None)
                    and np.all(rightEyePupils != None)):
                    pyautogui.click(button="left")
                elif (right_eye_ratio > blinking_ratio and right_eye_ratio > 5.7 and np.all(rightEyePupils == None)
                    and np.all(leftEyePupils != None)):
                    pyautogui.click(button="right")

except:
    pass
```

```

class MainWindow(QMainWindow):
    def __init__(self, *args, **kwargs):
        super(MainWindow, self).__init__(*args, **kwargs)

        self.title = "Dijital Gözüm"
        self.setWindowTitle(self.title)
        self.setWindowIcon(QIcon("./tools/cam.ico"))

        self.tab_widget = MyTabWidget(self)
        self.setCentralWidget(self.tab_widget)

class MyTabWidget(QWidget):
    def __init__(self, parent):
        super(QWidget, self).__init__(parent)
        self.layout = QVBoxLayout(self)

        self.tabs = QTabWidget()
        self.tab1 = QWidget()
        self.tab2 = QWidget()
        self.tab3 = QWidget()
        self.tab4 = QWidget()

        self.tabs.addTab(self.tab1, "Kamera")
        self.tabs.addTab(self.tab2, "Ayarlar")
        self.tabs.addTab(self.tab3, "Kısa yollar")
        self.tabs.addTab(self.tab4, "Nasıl Kullanılır")

        self.tab1.layout = QVBoxLayout(self)

        self.label_top = QLabel(self)
        self.pixmap = QPixmap('./tools/toppng.png')
        self.label_top.setPixmap(self.pixmap)

        self.label_bottom = QLabel(self)
        self.pixmap_bottom = QPixmap('./tools/toplogo.png')
        self.label_bottom.setPixmap(self.pixmap_bottom)

        self.CancelBTN = QPushButton("Çıkış")
        self.CancelBTN.clicked.connect(self.CancelFeed)

        self.FeedLabel = QLabel()
        self.pixmap_wait = QPixmap("./tools/camwait1.png")
        self.FeedLabel.setPixmap(self.pixmap_wait)
        self.FeedLabel.setStyleSheet("border: 3px solid blue;")

        self.tab1.layout.addWidget(self.label_top)
        self.tab1.layout.addWidget(self.FeedLabel)
        self.tab1.layout.addWidget(self.CancelBTN)
        self.tab1.layout.addWidget(self.label_bottom)

```

```
self.tab1.setLayout(self.tab1.layout)

self.tab2.layout = QVBoxLayout(self)
self.tab2.hlayout1 = QHBoxLayout(self)
self.tab2.hlayout2 = QHBoxLayout(self)

self.label_top2 = QLabel(self)
self.pixmap2 = QPixmap('./tools/toplogo.png')
self.label_top2.setPixmap(self.pixmap2)
self.tab2.layout.addWidget(self.label_top2)

self.margin_1 = QLabel(self)
self.margin_1.setStyleSheet("QLabel"
                             "{
                             \"border : 2px solid black;\"
                             \"background : white;\"
                             }")

self.margin_1.setAlignment(Qt.AlignCenter)

self.margin_1.setFont(QFont('Arial', 40))
self.margin_1.setText(f"Mod 1: {mod1X}")

self.push_plus_1 = QPushButton("+")
self.push_plus_1.clicked.connect(self.push_plus1_f)

self.push_minus_1 = QPushButton("-")
self.push_minus_1.clicked.connect(self.push_minus1_f)

self.margin_2 = QLabel(self)
self.margin_2.setStyleSheet("QLabel"
                             "{
                             \"border : 2px solid black;\"
                             \"background : white;\"
                             }")

self.margin_2.setAlignment(Qt.AlignCenter)
```

```

self.margin_2.setFont(QFont('Arial', 40))
self.margin_2.setText(f"Mod 2: {mod2X}")

self.push_plus_2 = QPushButton("+")
self.push_plus_2.clicked.connect(self.push_plus2_f)

self.push_minus_2 = QPushButton("-")
self.push_minus_2.clicked.connect(self.push_minus2_f)

self.tab2.hlayout1.addWidget(self.push_minus_1)
self.tab2.hlayout1.addWidget(self.push_plus_1)

self.tab2.hlayout2.addWidget(self.push_minus_2)
self.tab2.hlayout2.addWidget(self.push_plus_2)

self.tab2.layout.addWidget(self.margin_1)
self.tab2.layout.addLayout(self.tab2.hlayout1)
self.tab2.layout.addWidget(self.margin_2)
self.tab2.layout.addLayout(self.tab2.hlayout2)

self.tab2.setLayout(self.tab2.layout)

self.tab3.layout = QVBoxLayout(self)

self.label_top3 = QLabel(self)
self.pixmap3 = QPixmap('./tools/shortcuts.png')
self.label_top3.setPixmap(self.pixmap3)
self.tab3.layout.addWidget(self.label_top3)

self.tab3.setLayout(self.tab3.layout)

self.tab4.layout = QVBoxLayout(self)

self.label_bottom4 = QLabel(self)
self.pixmap_bottom4 = QPixmap('./tools/bottompng.png')
self.label_bottom4.setPixmap(self.pixmap_bottom4)

self.label_top4 = QLabel(self)
self.pixmap_top4 = QPixmap('./tools/use.png')
self.label_top4.setPixmap(self.pixmap_top4)

self.label_top41 = QLabel(self)
self.pixmap_top41 = QPixmap('./tools/use1.png')
self.label_top41.setPixmap(self.pixmap_top41)

```

```

self.tab4.layout.addWidget(self.label_top4)
self.tab4.layout.addWidget(self.label_top41)
self.tab4.layout.addWidget(self.label_bottom4)

self.tab4.setLayout(self.tab4.layout)

self.Worker1 = Worker1()

self.Worker1.start()
self.Worker1.ImageUpdate.connect(self.ImageUpdateSlot)

self.layout.addWidget(self.tabs)
self.setLayout(self.layout)

self.show()

def ImageUpdateSlot(self, Image):
    self.FeedLabel.setPixmap(QPixmap.fromImage(Image))

def CancelFeed(self):
    self.Worker1.stop()
    global a
    a += 1

def push_plus1_f(self):
    global mod1X
    mod1X += 1
    if mod1X >= 100:
        mod1X = 100
        self.margin_1.setText(f"Mod 1: {mod1X}")
    self.margin_1.setText(f"Mod 1: {mod1X}")

def push_minus1_f(self):
    global mod1X
    mod1X -= 1
    if mod1X <= 0:
        mod1X = 0
        self.margin_1.setText(f"Mod 1: {mod1X}")
    self.margin_1.setText(f"Mod 1: {mod1X}")

def push_plus2_f(self):
    global mod2X
    mod2X += 1
    if mod2X >= 100:
        mod2X = 100
        self.margin_2.setText(f"Mod 2: {mod2X}")
    self.margin_2.setText(f"Mod 2: {mod2X}")

def push_minus2_f(self):
    global mod2X
    mod2X -= 1
    if mod2X <= 0:
        mod2X = 0
        self.margin_2.setText(f"Mod 2: {mod2X}")
    self.margin_2.setText(f"Mod 2: {mod2X}")

class Worker1(QThread):
    ImageUpdate = pyqtSignal(QImage)

    def run(self):
        self.ThreadActive = True

        if ret:
            Image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            ConvertToQtFormat = QImage(Image.data, Image.shape[1], Image.shape[0], QImage.Format_RGB888)
            Pic = ConvertToQtFormat.scaled(640, 480, Qt.KeepAspectRatio)
            self.ImageUpdate.emit(Pic)
        except:
            pass

if __name__ == "__main__":
    App = QApplication(sys.argv)
    Root = MainWindow()
    Root.show()
    sys.exit(App.exec_())

```

4

5

6